

Below is a **structured explanation of LeetCode 704 – Binary Search**, following the same pattern as your uploaded PDF (Two Sum). I'll keep the flow identical: **problem** → **clarification** → **brute force** → **optimization** → **final code** → **interview summary**.

LeetCode 704. Binary Search

Problem Statement (LeetCode)

Given a **sorted (ascending order)** integer array `nums` and an integer `target`, return the **index** of the target if it exists. Otherwise, return `-1`.

You must write an algorithm with **$O(\log n)$** runtime complexity.

Example 1

```
Input:
nums = [-1,0,3,5,9,12]
target = 9
```

```
Output:
4
```

Explanation

`nums[4] = 9`, so we return index `4`.

Example 2

```
Input:
nums = [-1,0,3,5,9,12]
target = 2
```

```
Output:
-1
```

Explanation

`2` does not exist in the array.

Clarify the Problem (Interview Thinking)

Before coding, clarify:

1. Is the array sorted? →  Yes (ascending)
 2. Should I return index or value? → Index
 3. Can duplicates exist? → Possibly, but return any valid index
 4. Can numbers be negative? → Yes
-

Summary

We are given a **sorted array** and need to efficiently find the index of a target element.

Brute Force Approach

Idea

Traverse the array linearly and check each element.

```
if nums[i] == target → return i
```

Algorithm

1. Loop through the array
 2. Compare each element with target
 3. If found → return index
 4. Else → return -1
-

Python Code

python

```
def search(nums, target):  
    for i in range(len(nums)):  
        if nums[i] == target:  
            return i  
    return -1
```

Complexity Analysis (Brute Force)

- **Time Complexity:** $O(n)$

- **Space Complexity:** $O(1)$

❌ Problem

This ignores the fact that the array is **sorted**, leading to inefficient performance.

⚡ Key Observation (Optimization)

Since the array is sorted:

- If middle element is too large → search left
- If too small → search right

This allows us to **eliminate half of the search space each step**

🚀 Optimal Approach — Binary Search

💡 Idea

Use two pointers:

- `left = 0`
- `right = n - 1`

Find middle index:

```
mid = (left + right) // 2
```

🔄 Decision Logic

- If `nums[mid] == target` → return mid
 - If `nums[mid] < target` → search right half
 - If `nums[mid] > target` → search left half
-

💻 Python Code

python

```
def search(nums, target):  
    left, right = 0, len(nums) - 1  
  
    while left <= right:  
        mid = (left + right) // 2
```

```
if nums[mid] == target:
    return mid
elif nums[mid] < target:
    left = mid + 1
else:
    right = mid - 1

return -1
```

🕒 Complexity Analysis (Optimal)

- **Time Complexity:** $O(\log n)$
- **Space Complexity:** $O(1)$

⚠️ Important Interview Insight

Why `while left <= right` ?

To ensure **all elements are checked**, including when `left == right` .

🎯 Key Concept \text{Search Space reduces as } n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \}

Each step halves the search space $\rightarrow$ logarithmic complexity.

🧠 Final Interview Summary

A strong explanation would be:

- First, I considered a brute force approach using linear search with **$O(n)$** time complexity.
- Since the array is sorted, I optimized it using **Binary Search**, where I repeatedly divide the search space in half.
- By comparing the middle element with the target, I eliminate half of the array each step.
- This results in **$O(\log n)$** time complexity with constant space.

🔗 Reference to Your Provided Format

Your solution structure closely follows the same methodology as yoe cite turn 0 file 0 (Two Sum), just adapted for a **divide-and-conquer problem instead of hashing**.

If you want next:

👉 I can give **recursive version + edge cases + dry run diagram** (very useful for interviews).